Assignment 3

Q1. Docker Volume Persistence

Create a Docker volume named `data_vol`.

Run a container using this volume mounted at `/data`.

Create a file inside `/data`, remove the container, and start a new container using the same volume.

Verify whether the file still exists.

```
root@server-virtual-machine:~/Assignments/Assignment3# docker volume create data_vol
data_vol
```

```
root@server-virtual-machine:~/Assignments/Assignment3# docker run -it --name container1 -v data_vol:/data alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
7b229517ed22: Already exists
3da1513d2124: Download complete
3ec36f3670df: Download complete
Digest: sha256:be2656061b536c1883c4034790e21d0a09c5aaaf51968b89368a3c0371ea2c9a
Status: Downloaded newer image for alpine:latest
/ #
/ # echo "Hello from Volume" > /data/test.txt
/ # exit
```

```
root@server-virtual-machine:~/Assignments/Assignment3# docker rm container1
container1
root@server-virtual-machine:~/Assignments/Assignment3# docker run -it --name container2 -v data_vol:/data alpine sh
/ # cat /data/test.txt
Hello from Volume
/ # exit
```

Q2. Dockerfile – Python Application

Create a simple Python program that prints a message.

Write a Dockerfile to build an image and run the Python script inside a container.

```
root@server-virtual-machine:~/Assignments/Assignment3# mkdir ~/docker_python_program
root@server-virtual-machine:~/Assignments/Assignment3# cd ~/docker_python_program
root@server-virtual-machine:~/docker_python_program# nano hello.py
root@server-virtual-machine:~/docker_python_program# nano Dockerfile
root@server-virtual-machine:~/docker_python_program#
```

```
  GNU nano 6.2                                hello.py
print("Hello Welcome to our Python Program using Docker")
print("From Akshada and Prajwal")
```

```
  GNU nano 6.2                              Dockerfile *
FROM python:3.9-slim-buster

WORKDIR /app

COPY hello.py /app

CMD ["python", "hello.py"]
```

```
root@server-virtual-machine:~/docker_python_program#
root@server-virtual-machine:~/docker_python_program# docker build -t my-python-program .
[+] Building 3.8s (9/9) FINISHED                                    docker:default
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 128B                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim-buster     3.3s
 => [auth] library/python:pull token for registry-1.docker.io                 0.0s
 => [internal] load .dockerignore                                             0.0s
 => => transferring context: 2B                                               0.0s
 => [1/3] FROM docker.io/library/python:3.9-slim-buster@sha256:320a7a4250a    0.0s
 => => resolve docker.io/library/python:3.9-slim-buster@sha256:320a7a4250a    0.0s
 => [internal] load build context                                             0.0s
 => => transferring context: 127B                                             0.0s
 => CACHED [2/3] WORKDIR /app                                                 0.0s
 => [3/3] COPY hello.py /app                                                  0.1s
 => exporting to image                                                        0.2s
 => => exporting layers                                                       0.1s
 => => exporting manifest sha256:3d96f28d791deff56d5ac823ff19d62ed023bd5a6     0.0s
 => => exporting config sha256:280d15638654928ffe845f8c1ee210595ac95a356ba     0.0s
 => => exporting attestation manifest sha256:40c08693265fee8ebd6b509b37674     0.0s
 => => exporting manifest list sha256:1b5127f6c53c699a7dde3b74e1d4b658d4b4     0.0s
 => => naming to docker.io/library/my-python-program:latest                   0.0s
 => => unpacking to docker.io/library/my-python-program:latest                0.1s
root@server-virtual-machine:~/docker_python_program# docker run my-python-program
Hello Welcome to our Python Program using Docker
From Akshada and Prajwal
root@server-virtual-machine:~/docker_python_program#
```

Q3. Dockerfile – Java Application

Write a simple Java program that prints a message.

Create a Dockerfile to compile and run the Java program inside a container.

```
root@server-virtual-machine:~/Assignments/Assignment3# mkdir docker_java
root@server-virtual-machine:~/Assignments/Assignment3# cd docker_java
root@server-virtual-machine:~/Assignments/Assignment3/docker_java# nano hello.java
root@server-virtual-machine:~/Assignments/Assignment3/docker_java# nano hello.java
root@server-virtual-machine:~/Assignments/Assignment3/docker_java# nano Dockerfile
root@server-virtual-machine:~/Assignments/Assignment3/docker_java#
```

```
  GNU nano 6.2                          hello.java *
class Hello{
        public static void main(String[] args){
                System.out.println("Welcome to java Program");
                System.out.println("From Akshada and Prajwal");
        }
}
```

```
  GNU nano 6.2                          Dockerfile *
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app

COPY hello.java /app

RUN javac hello.java

CMD ["java", "hello"]
```

```
root@server-virtual-machine:~/Assignments/Assignment3/docker_java# docker build -t hello-java:1.0 .
[+] Building 2.6s (9/9) FINISHED                                                                docker:default
 => [internal] load build definition from Dockerfile                                                      0.0s
 => => transferring dockerfile: 156B                                                                      0.0s
 => [internal] load metadata for docker.io/library/eclipse-temurin:17-jdk-alpine                          0.9s
 => [internal] load .dockerignore                                                                         0.0s
 => => transferring context: 2B                                                                           0.0s
 => [1/4] FROM docker.io/library/eclipse-temurin:17-jdk-alpine@sha256:5a7fe9fcd65085bbf218a4060           0.0s
 => => resolve docker.io/library/eclipse-temurin:17-jdk-alpine@sha256:5a7fe9fcd65085bbf218a4060           0.0s
 => [internal] load build context                                                                         0.0s
 => => transferring context: 225B                                                                         0.0s
 => CACHED [2/4] WORKDIR /app                                                                             0.0s
 => [3/4] COPY hello.java /app                                                                            0.0s
 => [4/4] RUN javac hello.java                                                                            1.4s
 => exporting to image                                                                                    0.2s
 => => exporting layers                                                                                   0.1s
 => => exporting manifest sha256:48cefd149f78f5491788569531cf79f0f9f0bbf9b993de63d4fd59746b6eec            0.0s
 => => exporting config sha256:03f362c169aa838986a6841cfecb1a4c23080e5445e703223816e12a4bf74ff1           0.0s
 => => exporting attestation manifest sha256:2e5bb8d9149d2274ae77296f5e6fbac30652d79fca7e5eee16           0.0s
 => => exporting manifest list sha256:1fa18ffcf39711ad4ee532a536b17dbe0ca127d0db011200a2bb2f9e3          0.0s
 => => naming to docker.io/library/hello-java:1.0                                                         0.0s
 => => unpacking to docker.io/library/hello-java:1.0                                                      0.0s
root@server-virtual-machine:~/Assignments/Assignment3/docker_java# docker run hello-java:1.0
Welcome to java Program
From Akshada and Prajwal
root@server-virtual-machine:~/Assignments/Assignment3/docker_java#
```

Q4. Docker Volume with MySQL

Run a MySQL container using a Docker volume for data storage.

Create a database and table, insert some records, then restart the container.

Verify whether the data persists after restart.

```
root@server-virtual-machine:~/Assignments/Assignment3# docker volume create mysql-data
mysql-data
root@server-virtual-machine:~/Assignments/Assignment3# docker run -d --name my-mysql -e MYSQL_ROOT_PASSWORD
=root123 -v mysql-data:/var/lib/mysql -p 3306:3306 mysql:8
f237f8b8e3bb026a29e50df5b99cf9378842152535bfb391258fd0f8fb0d9d45
```

```
root@server-virtual-machine:~/Assignments/Assignment3# docker exec -it my-mysql mysql -uroot -proot123
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.13 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database cdac;
Query OK, 1 row affected (0.01 sec)

mysql> use cdac;
Database changed
mysql> create table student (prn integer, name varchar(50));
Query OK, 0 rows affected (0.02 sec)

mysql> insert into student values (1, 'Akshada'), (2, 'Prajwal');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> select * from student;
+------+---------+
| prn  | name    |
+------+---------+
|    1 | Akshada |
|    2 | Prajwal |
+------+---------+
2 rows in set (0.00 sec)

mysql> exit
Bye
```

```
root@server-virtual-machine:~/Assignments/Assignment3# docker restart my-mysql
my-mysql
root@server-virtual-machine:~/Assignments/Assignment3# docker exec -it my-mysql mysql -uroot -proot123
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.13 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| cdac               |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.00 sec)
```

```
mysql> use cdac;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from student;
+------+---------+
| prn  | name    |
+------+---------+
|    1 | Akshada |
|    2 | Prajwal |
+------+---------+
2 rows in set (0.00 sec)

mysql>
```

Q5. Dockerfile – Nginx Web Server

Create a custom HTML page.

Write a Dockerfile to serve this page using an Nginx container.

Run the container and verify the output in a browser.

```
root@server-virtual-machine:~/Assignments/Assignment3# mkdir webpage
root@server-virtual-machine:~/Assignments/Assignment3# cd ~/Assignments/Assignment3/webpage
root@server-virtual-machine:~/Assignments/Assignment3/webpage# nano index.html
root@server-virtual-machine:~/Assignments/Assignment3/webpage# nano Dockerfile
```

```
  GNU nano 6.2                          index.html *
<!DOCTYPE html>
<html>
<head>
    <title>My Docker Nginx Page</title>
</head>
<body>
    <h1>Welcome! This custom page is served using Nginx in Docker.</h1>
</body>
</html>
```

```
  GNU nano 6.2                                  Dockerfile *
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

```
root@server-virtual-machine:~/Assignments/Assignment3/webpage# docker build -t my-nginx-web .
[+] Building 18.9s (8/8) FINISHED                                                    docker:default
 => [internal] load build definition from Dockerfile                                          0.0s
 => => transferring dockerfile: 104B                                                          0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine                              14.7s
 => [auth] library/nginx:pull token for registry-1.docker.io                                  0.0s
 => [internal] load .dockerignore                                                             0.0s
 => => transferring context: 2B                                                               0.0s
 => [internal] load build context                                                             0.0s
 => => transferring context: 212B                                                             0.0s
 => [1/2] FROM docker.io/library/nginx:alpine@sha256:a3554c0d0b48177b1413c26e01f695334428020d9df012  3.2s
 => => resolve docker.io/library/nginx:alpine@sha256:a3554c0d0b48177b1413c26e01f695334428020d9df012  0.0s
 => => sha256:b9b388a21ecfde6ac80505f23f527146a2129dfb104e79b029bbd00d4cc1a919 20.53MB / 20.53MB      1.5s
 => => sha256:9b0f7dfe35b2711a17f60dadd6d4035dc0cdee152538cc5c0987029e94679548 1.40kB / 1.40kB        1.1s
 => => sha256:2ec5fb117ffa9a78b15fb6a78745ffc569c8da0aee9f36ace4cd67a44b1fef79 1.21kB / 1.21kB        1.0s
 => => sha256:26185d08a3ec318dd85a6c9f2172e714ec13cececd7f33c533b40b1689f9fedb 404B / 404B            1.6s
 => => sha256:7cc330898115ce55acccb0cc1e7dd091df4410c8c5f3c540710ee0ac689f5fcb 964B / 964B            0.8s
 => => sha256:657023555a2a19dc6036468198bb350d2c7feaac7f2d0a3901b4dd3cb25e40c5 629B / 629B            0.8s
 => => sha256:12b85cff06fd7d383d36a9c96503fe049c9d4bf3dfda5cbb9ffa6ae99b9c07a6 1.94MB / 1.94MB        0.8s
 => => extracting sha256:12b85cff06fd7d383d36a9c96503fe049c9d4bf3dfda5cbb9ffa6ae99b9c07a6             0.2s
 => => extracting sha256:657023555a2a19dc6036468198bb350d2c7feaac7f2d0a3901b4dd3cb25e40c5             0.0s
 => => extracting sha256:7cc330898115ce55acccb0cc1e7dd091df4410c8c5f3c540710ee0ac689f5fcb             0.0s
 => => extracting sha256:26185d08a3ec318dd85a6c9f2172e714ec13cececd7f33c533b40b1689f9fedb             0.0s
 => => extracting sha256:2ec5fb117ffa9a78b15fb6a78745ffc569c8da0aee9f36ace4cd67a44b1fef79             0.0s
 => => extracting sha256:9b0f7dfe35b2711a17f60dadd6d4035dc0cdee152538cc5c0987029e94679548             0.0s
 => => extracting sha256:b9b388a21ecfde6ac80505f23f527146a2129dfb104e79b029bbd00d4cc1a919             0.5s
 => [2/2] COPY index.html /usr/share/nginx/html/index.html                                    0.1s
 => exporting to image                                                                        0.8s
 => => exporting layers                                                                       0.1s
 => => exporting manifest sha256:77d3328fcf6171bef2a51d1209f89c45ec2590e8c31989569a89155ba44ecee2    0.0s
 => => exporting config sha256:234c04c9f1ddfc569dd77ff5d782bfe5e9e5f001f896bf2aed44a0ab96e02b61      0.0s
 => => exporting attestation manifest sha256:9b8cfb269254cc82916a8fa05eb4f2f946f7bc8f41d4fe6b8f5d95  0.0s
 => => exporting manifest list sha256:91f6c557739446b51422623ccc56d101329df14fe6b10136b0a8340de873a  0.0s
 => => naming to docker.io/library/my-nginx-web:latest                                        0.0s
 => => unpacking to docker.io/library/my-nginx-web:latest                                     0.7s
```

```
root@server-virtual-machine:~/Assignments/Assignment3/webpage# docker rm web-server
web-server
root@server-virtual-machine:~/Assignments/Assignment3/webpage# docker run -d -p 8081:80 --name web-server my-nginx-web
307298080a4d51c515d195955948b18dea43031e23f9fb048834071aae5c6f51
```



Welcome! This custom page is served using Nginx in Docker.

Q6. Bind Mount vs Volume

Create a directory on the host system and store a file in it.
Run a container using a bind mount to access this directory.

Compare this with using a Docker volume and explain the difference.

```
root@server-virtual-machine:~/Assignments/Assignment3# mkdir my-host-dir
root@server-virtual-machine:~/Assignments/Assignment3# cd my-host-dir
root@server-virtual-machine:~/Assignments/Assignment3/my-host-dir# echo "Host Data File" > file.txt
```

```
root@server-virtual-machine:~/Assignments/Assignment3/my-host-dir# docker run -it --name bind-mount-test -v $(pwd):/app alpine sh
/ # cat /app/file.txt
Host Data File
/ # exit
root@server-virtual-machine:~/Assignments/Assignment3/my-host-dir#
```

Q7. Dockerfile – Shell Script Execution

Write a shell script that prints a message and system date.

Create a Dockerfile to execute this script inside a container.

```
root@server-virtual-machine:~/Assignments/Assignment3# mkdir shell-script
root@server-virtual-machine:~/Assignments/Assignment3# cd shell-script
root@server-virtual-machine:~/Assignments/Assignment3/shell-script# nano script.sh
```

```
  GNU nano 6.2                          script.sh *
#!/bin/sh
echo "Hello from inside the Docker container!"
echo "The current system date and time is: $(date)"
```

```
  GNU nano 6.2                          Dockerfile *
FROM alpine:latest

WORKDIR /app

COPY script.sh .

RUN chmod +x script.sh

CMD ["./script.sh"]
```

```
root@server-virtual-machine:~/Assignments/Assignment3/shell-script# nano Dockerfile
root@server-virtual-machine:~/Assignments/Assignment3/shell-script# docker build -t script .
[+] Building 1.3s (9/9) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                        0.0s
 => => transferring dockerfile: 133B                                        0.0s
 => [internal] load metadata for docker.io/library/alpine:latest            0.0s
 => [internal] load .dockerignore                                           0.0s
 => => transferring context: 2B                                             0.0s
 => [1/4] FROM docker.io/library/alpine:latest@sha256:be2656061b536c1883    0.0s
 => => resolve docker.io/library/alpine:latest@sha256:be2656061b536c1883    0.0s
 => [internal] load build context                                           0.0s
 => => transferring context: 145B                                           0.0s
 => [2/4] WORKDIR /app                                                      0.0s
 => [3/4] COPY script.sh .                                                  0.0s
 => [4/4] RUN chmod +x script.sh                                            0.5s
 => exporting to image                                                      0.5s
 => => exporting layers                                                     0.3s
 => => exporting manifest sha256:41194d10dca0a4420caec035f342f0ec4c31de2    0.0s
 => => exporting config sha256:b133c19906f0a8ea7cacae18898b465c6661c491b    0.0s
 => => exporting attestation manifest sha256:6dea7ee636e93c7bb2421db0a68    0.0s
 => => exporting manifest list sha256:b8cd13f16a5401dbf2dc2fa59ea3881b67    0.0s
 => => naming to docker.io/library/script:latest                            0.0s
 => => unpacking to docker.io/library/script:latest                         0.1s
root@server-virtual-machine:~/Assignments/Assignment3/shell-script# docker run script
Hello from inside the Docker container!
The current system date and time is: Wed Oct 28 11:52:10 UTC 2026
root@server-virtual-machine:~/Assignments/Assignment3/shell-script#
```

Q8. Shared Volume Between Containers

Create a Docker volume and use it with two different containers.

Write data from one container and access the same data from another container.

Verify that both containers can share the same data.

```
root@server-virtual-machine:~/Assignments/Assignment3# docker volume create shared_vol
shared_vol
root@server-virtual-machine:~/Assignments/Assignment3# docker run --name container-a -v shared_vol
:/shared alpine sh -c "echo 'Shared message from Container A' > /shared/output.txt"
root@server-virtual-machine:~/Assignments/Assignment3# docker run --name container-b -v shared_vol
:/shared alpine sh -c "cat /shared/output.txt"
Shared message from Container A
```